Lab 2.1 - Brownfield archaeology (Developers)

Goal

Build a map of the legacy code that you can trust, and find out where one calculation really lives. You will use three ways of navigating the code, compare them, and learn what each one misses.

You're done when

- ☐ `docs/notes/legacy-map.md` exists. Every line cites file + method. You checked 3 lines yourself and marked them ✓ or ✗
- ☐ The race table (step 3) is filled: what each method found, what it missed, and why
- ☐ You know every place that calculates kidney volume, and the number each one gives for SYN-0002
- ☐ `docs/notes/kidney-volume-decision.md` lists the options for the ignored test. You did not change any code

Option A - Guided

0. Get the code graph (2 min). Pick one:

```
pip install graphifyy
graphify update .
```

No Python, or not allowed to install? Copy the pre-built graph from its branch instead:

```
git checkout lab/2.1-graph -- graphify-out
git reset -q graphify-out
```

Either way, keep it out of your commits: add the line `graphify-out/` to `.git/info/exclude`.

1. Map the legacy code with subagents (10 min). New chat, **Agent** mode:

```
Map the legacy part of this solution. Use one subagent per project, in parallel:
src/OrganLens.Legacy.Core, src/OrganLens.Legacy.Data, src/OrganLens.Legacy.WinForms.
Each subagent reports: purpose, entry points, who calls it, where data comes from,
config switches, tests (including skipped ones), and smells.
Merge the reports into docs/notes/legacy-map.md.
Every line cites file + method. Write UNKNOWN when you could not confirm something.
Do not change any code.
```

If it does the work itself instead, stop it and say: `Start a separate subagent for each project.` Watch the main chat's context meter while the subagents work. The research happens in *their* context, not yours.

2. Trust, but verify (5 min). Pick 3 lines from the map. Open the cited files yourself. Mark each line ✓ (true) or ✗ (wrong or invented) in the file.

3. The race (15 min). One question, three methods:

> *Where is kidney volume calculated in the whole solution? List every place, with its formula and constant.*

	Method	How
A	Plain search	No AI. `Ctrl+Shift+F`, search for `Volume`. 3 minutes.
B	Symbols	**Ask** mode, prompt below
C	Code graph	Terminal commands below, then the agent with the graph

Prompt for B:

```
Start from KidneyVolumeCalculator.CalculateMl. Use #usages to find every caller.
Then look in the legacy projects for any other code that computes a kidney volume,
even if it does not call this method. List each place: file, method, line, constant.
```

Commands for C:

```
graphify query "where is kidney volume calculated"
graphify explain "src/OrganLens.Legacy.WinForms/MainForm.cs::MainForm.btnAnalyze_Click()"
```

Then **Ask** mode:

```
Read graphify-out/GRAPH_REPORT.md. Using only the graph (you may run graphify query
and graphify explain in the terminal), answer: where is kidney volume calculated?
Then tell me what kinds of facts a graph like this can NOT show.
```

Fill in the table:

Method	Places found	Wrong claims	Time	Why it missed what it missed
A Plain search				
B Symbols				
C Graph				

4. Do the arithmetic (3 min). SYN-0002 is 110 × 50 × 45 mm. Work out by hand (or with a calculator) what each place you found returns. Compare with what the agent says.

5. Git archaeology (5 min). Agent mode:

```
Use git log -S and git blame to find who introduced each kidney volume constant, and why.
Also explain the commit that added [Ignore] to KidneyVolume_MatchesWebAnalysis.
If the history does not tell you, say so. Do not guess a reason.
```

Did it guess? A good answer here can be "unknown, ask a person".

6. Decide, don't fix (5 min). Ask mode:

```
The test KidneyVolume_MatchesWebAnalysis is ignored as "flaky".
Is it really flaky? Give evidence. Then list 3 options (for example: fix the legacy formula,
change the test, document the difference), each with its risk for a medical product.
Write it to docs/notes/kidney-volume-decision.md. Do not change any code.
```

The decision belongs to a person. Leave it as an open question for your team lead.

7. Keep the best prompt. Save your step 1 prompt as `.github/prompts/map-module.prompt.md`, with the project as an input:

```
---
description: Map one legacy project with citations
---
Map ${input:project}. Report purpose, entry points, callers, data sources,
config switches, tests (including skipped) and smells.
Cite file + method for every line. Write UNKNOWN when not confirmed. Do not change code.
```

Try it: type `/map-module` in chat.

> Visual Studio: Find All References (`Shift+F12`) is your symbol tool. Copilot Chat in Visual Studio reads the same prompt files.

Option B - Challenge

Same results, your own prompts. Rules:

- Start from the graph: `GRAPH_REPORT.md` first, then decide where to dig.
- Your map must also find at least 3 risks that are not about kidneys. One of them must be a security issue that is switched off today.

- For every place the graph missed, explain why it could not have found it.
- Keep the main chat under 50% of the context window. Show the meter at the end.

Stop and check

- Which method found the copy inside a button click handler? Why did the others miss it?
- How many of your 3 checked map lines were wrong?
- Would you trust the graph alone for "what breaks if I change this method"? Why not?